

SOFTWARE REQUIREMENTS SPECIFICATION

ATM DemandIQ

AI-Powered ATM Cash Demand Forecasting & Replenishment Optimization System

Version: 1.0

Project Type: Data Analytics & Machine Learning

Domain: Banking / FinTech

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of ATM DemandIQ, an AI-powered system designed to forecast cash demand at ATMs.

The system analyzes historical ATM transaction and cash withdrawal data to identify demand patterns and predict the amount of cash likely to be required at a particular ATM for a future period.

The system aims to assist banks and ATM operators in making data-driven cash replenishment decisions, reducing the possibility of ATM cash-outs while avoiding unnecessary excess cash holding.

1.2 Project Overview

ATM DemandIQ combines data analytics, machine learning, backend APIs, database management, and an interactive dashboard to provide ATM cash-demand forecasts.

The overall workflow is:

Data Collection → Preprocessing → EDA → Feature Engineering → Model Development → Model Optimization → API Integration → Forecasting → Results

2. Problem Statement

ATM cash management is an important operational challenge for banks and financial institutions.

ATM cash demand varies depending on several factors, including:

- **Day of the week**
- **Month and season**
- **Holidays**
- **ATM location**
- **Historical withdrawal patterns**
- **Transaction volume**
- **Salary/payment cycles**
- **Special events**

If an ATM runs out of cash, customers may be unable to withdraw money. On the other hand, loading excessive cash can result in inefficient utilization of available cash.

ATM DemandIQ aims to use historical data and machine learning techniques to forecast future ATM cash requirements and provide insights for cash replenishment planning.

3. Objectives

The primary objectives of the system are:

- 1. Collect historical ATM transaction data.**
 - 2. Clean and preprocess the collected data.**
 - 3. Perform exploratory data analysis.**
 - 4. Identify cash-demand trends and patterns.**
 - 5. Engineer relevant prediction features.**
 - 6. Develop machine learning/forecasting models.**
 - 7. Optimize the selected model.**
 - 8. Generate ATM-wise cash-demand predictions.**
 - 9. Evaluate prediction performance.**
 - 10. Provide an interactive dashboard.**
 - 11. Identify potential cash-out situations.**
 - 12. Provide data-driven replenishment insights.**
-

4. Scope of the System

4.1 In-Scope

The system will include:

- **Historical ATM data ingestion**

- Data validation
- Data cleaning
- Missing-value handling
- Duplicate removal
- Outlier detection
- Feature engineering
- Exploratory data analysis
- ATM-wise demand analysis
- Forecast model development
- Model comparison
- Hyperparameter optimization
- Prediction generation
- Model evaluation
- REST API integration
- Dashboard visualization
- Replenishment recommendations
- Analytical reports

4.2 Out-of-Scope

The initial version will not include:

- Direct ATM hardware control
- Physical cash replenishment
- Real-time transaction authorization
- ATM firmware integration
- Direct financial transactions
- Automated physical cash delivery

The system will provide predictions and recommendations, while actual cash replenishment decisions will remain with authorized personnel.

5. User Types

5.1 Administrator

The administrator can:

- Manage users
- Configure system settings
- Monitor system status
- Manage datasets
- View system-level reports

5.2 Cash Management User

The user can:

- View ATM forecasts
- Search individual ATMs
- View historical demand
- View predicted cash requirements
- Monitor potential cash-out situations
- View replenishment recommendations
- Download reports

5.3 Data Analyst

The analyst can:

- Upload datasets
- Validate data
- Perform EDA
- View statistical summaries
- Compare models
- Analyze prediction errors

6. EPIC Structure

The project is divided into four major EPICs:

EPIC	Name	Objective
EPIC 1	Data Collection & Preprocessing	Prepare clean and reliable data
EPIC 2	Exploratory Data Analysis	Identify trends and demand patterns
EPIC 3	Model Optimization & Pipeline Integration	Build, optimize and integrate the model
EPIC 4	Results & Prediction Accuracy	Evaluate predictions and generate insights

7. EPIC 1 — Data Collection & Preprocessing

FR-01: Dataset Upload

The system shall allow authorized users to upload historical ATM transaction datasets.

Supported formats:

- CSV
- Excel

FR-02: Data Validation

The system shall validate uploaded datasets for:

- Required columns
- Data types
- Missing values
- Invalid dates
- Duplicate records
- Invalid numerical values

FR-03: Missing Value Handling

The system shall identify missing values and apply suitable techniques such as:

- Mean/median imputation
- Forward filling
- Backward filling
- Removal where appropriate

FR-04: Duplicate Removal

The system shall identify and remove duplicate transaction records.

FR-05: Outlier Detection

The system shall identify unusual transaction and cash-demand values using appropriate statistical or machine-learning techniques.

FR-06: Data Transformation

The system shall transform raw data into a format suitable for analysis and model training.

FR-07: Feature Engineering

The system shall generate features such as:

- Date
- Day
- Week
- Month
- Year
- Day of week
- Weekend indicator
- Holiday indicator
- ATM ID
- Previous-day demand
- Rolling average demand
- Lag features

FR-08: Dataset Splitting

The system shall divide the dataset into training, validation, and testing sets using time-aware splitting where required.

8. EPIC 2 — Exploratory Data Analysis

FR-09: Descriptive Statistics

The system shall calculate:

- Mean
- Median
- Mode
- Minimum
- Maximum
- Standard deviation
- Percentiles

FR-10: Demand Trend Analysis

The system shall display cash-demand trends over:

- Daily periods
- Weekly periods
- Monthly periods
- Yearly periods

where sufficient data is available.

FR-11: ATM-Wise Analysis

The system shall allow users to compare demand between different ATMs.

FR-12: Weekday Analysis

The system shall analyze differences between weekdays and weekends.

FR-13: Seasonal Analysis

The system shall identify recurring seasonal patterns in ATM demand.

FR-14: Holiday Analysis

Where holiday information is available, the system shall analyze the effect of holidays on cash demand.

FR-15: Correlation Analysis

The system shall identify relationships between relevant features and cash demand.

FR-16: Data Visualization

The system shall provide:

- Line charts
 - Bar charts
 - Histograms
 - Box plots
 - Heatmaps
 - Actual vs predicted charts
-

9. EPIC 3 — Model Optimization & Pipeline Integration

FR-17: Model Training

The system shall train one or more suitable forecasting or machine-learning models.

Potential models include:

- Linear Regression
- Random Forest
- Gradient Boosting
- XGBoost/LightGBM, if used
- ARIMA/SARIMA
- Other suitable forecasting models

FR-18: Model Comparison

The system shall compare candidate models using appropriate evaluation metrics.

FR-19: Hyperparameter Optimization

The system shall optimize model parameters using techniques such as:

- Grid Search
- Random Search
- Time-series validation

FR-20: Feature Selection

The system shall identify features that contribute meaningfully to prediction performance.

FR-21: Prediction Pipeline

The system shall implement the following pipeline:

Input Data → Validation → Preprocessing → Feature Engineering → Model → Prediction

FR-22: Model Persistence

The trained model shall be saved so that it can be reused without retraining for every prediction request.

FR-23: API Integration

The system shall expose prediction functionality through REST APIs using FastAPI.

FR-24: Prediction Request

The API shall accept relevant ATM and historical information and return the predicted cash demand.

FR-25: Frontend Integration

The frontend shall communicate with the backend prediction API and display the results.

10. EPIC 4 — Results & Prediction Accuracy

FR-26: Demand Forecast

The system shall generate future ATM cash-demand predictions.

FR-27: Actual vs Predicted Comparison

The system shall allow users to compare actual cash demand against predicted demand.

FR-28: Model Evaluation

The system shall calculate appropriate evaluation metrics, including:

- MAE
- RMSE
- MAPE, where appropriate
- R^2 , where appropriate

FR-29: Accuracy Dashboard

The system shall display model-performance metrics through the dashboard.

FR-30: Error Analysis

The system shall identify ATMs or periods where prediction errors are relatively high.

FR-31: Replenishment Recommendation

Based on forecast demand and configurable operational thresholds, the system may generate a suggested replenishment requirement.

FR-32: Alert Generation

The system may generate alerts for:

- Predicted high demand
- Potential cash-out risk
- Unusual demand spikes
- Low available cash

FR-33: Report Generation

The system shall allow users to generate and download analytical reports.

11. Non-Functional Requirements

11.1 Performance

- The prediction API should respond within an acceptable time under normal load.
- Dashboard visualizations should load efficiently.
- Data processing should remain reasonable for the expected dataset size.

11.2 Scalability

The system should support:

- Additional ATMs
- Larger datasets
- Additional users
- Additional forecasting models
- Future real-time data integration

11.3 Reliability

The system should:

- Handle invalid input gracefully.
- Prevent application crashes due to malformed datasets.
- Provide meaningful error messages.
- Produce consistent results for the same model and input conditions.

11.4 Security

The system should implement:

- Authentication
- Role-based access control
- Secure API endpoints
- Password protection
- Secure database access
- Input validation

11.5 Usability

The dashboard should:

- Be easy to understand.
- Use clear labels.
- Present important KPIs clearly.
- Provide intuitive navigation.
- Clearly distinguish historical and predicted values.

11.6 Maintainability

The system should use modular components so that:

- Models can be replaced.
 - Features can be modified.
 - APIs can be extended.
 - Database structures can evolve.
 - Frontend components can be maintained independently.
-

12. System Architecture

The proposed architecture is:

ATM Transaction Data



Data Collection & Validation



Data Preprocessing



Feature Engineering



Exploratory Data Analysis



ML / Forecasting Model



Model Optimization



FastAPI Backend



React Dashboard



Forecast Results



Replenishment Insights

13. Data Requirements

The system may require the following fields:

Field	Description	Type
ATM_ID	Unique ATM identifier	String
Date	Transaction/demand date	Date
Location	ATM location	String
Cash_Withdrawn	Total cash withdrawn	Numeric
Transactions	Number of transactions	Integer
Available_Cash	Cash available	Numeric
Replenishment	Cash replenished	Numeric
Holiday	Holiday indicator	Boolean
Day_of_Week	Day of week	Categorical

Additional fields may be added depending on dataset availability.

14. Inputs and Outputs

14.1 Inputs

The system may receive:

- Historical ATM transaction data
- ATM identifiers
- Dates
- Cash withdrawal amounts
- Transaction counts
- Available cash
- Replenishment data
- Holiday information
- User prediction requests

14.2 Outputs

The system shall produce:

- Predicted cash demand
- Model-performance metrics
- Historical trend charts
- Actual vs predicted charts
- ATM-wise analysis
- High-demand alerts
- Cash-out risk indicators
- Replenishment recommendations
- Analytical reports

15. Database Requirements

A relational database such as PostgreSQL/MySQL may be used.

15.1 ATM Table

Column	Description
ATM_ID	Primary key
Location	ATM location
ATM_Type	ATM category
Status	Active/Inactive

15.2 Transactions Table

Column	Description
Transaction_ID	Primary key
ATM_ID	Foreign key
Transaction_Date	Transaction date
Withdrawal_Amount	Amount withdrawn
Transaction_Count	Number of transactions

15.3 Cash Inventory Table

Column	Description
--------	-------------

Inventory_ID	Primary key
ATM_ID	Foreign key
Date	Record date
Opening_Cash	Opening balance
Replenished_Cash	Cash added
Closing_Cash	Closing balance

15.4 Predictions Table

Column	Description
Prediction_ID	Primary key
ATM_ID	Foreign key
Prediction_Date	Forecast date
Predicted_Demand	Forecasted demand
Model_Version	Model used
Created_At	Timestamp

16. API Requirements

The backend will expose REST APIs.

Method	Endpoint	Purpose
POST	/api/upload	Upload dataset
GET	/api/atms	Retrieve ATM list
GET	/api/atm/{id}/history	Retrieve ATM history
GET	/api/atm/{id}/forecast	Retrieve forecast
POST	/api/predict	Generate prediction
GET	/api/model/metrics	Retrieve model metrics
GET	/api/dashboard	Retrieve dashboard data
GET	/api/alerts	Retrieve alerts

17. User Interface Requirements

Dashboard

The dashboard should display:

- Total ATMs
- Forecasted total demand
- High-demand ATMs
- Potential cash-out risks
- Model performance
- Recent predictions

ATM Analysis

Users should be able to view:

- Historical demand
- Forecast demand
- Transaction volume
- Demand trends
- Replenishment history

Forecast Page

The forecast page should display:

- Forecast date
- Predicted demand
- Confidence/uncertainty information where supported
- Historical comparison

Model Performance

The page should display:

- MAE
- RMSE
- MAPE where appropriate
- R^2 where appropriate
- Model comparison

Alerts

The system should display:

- High-demand alerts
 - Potential cash-out alerts
 - Abnormal demand alerts
-

18. Use Cases

UC-01: Upload Dataset

Actor: Data Analyst

Steps:

1. User opens the upload page.
2. User selects a CSV/Excel file.
3. System validates the file.
4. System checks required columns.
5. System processes the dataset.
6. System displays validation results.

Postcondition: Valid data is prepared for analysis.

UC-02: View ATM Forecast

Actor: Cash Management User

Steps:

1. User selects an ATM.
2. User selects forecast period.
3. System retrieves required data.
4. Model generates or retrieves the forecast.
5. System displays predicted cash demand.

UC-03: Analyze ATM Performance

Actor: Data Analyst

Steps:

1. User selects an ATM.
2. System retrieves historical data.
3. System calculates statistics.
4. System generates visualizations.
5. User analyzes demand patterns.

UC-04: View Replenishment Recommendation

Actor: Cash Management User

Steps:

1. User selects an ATM.
2. System retrieves predicted demand.

3. System considers available cash and operational thresholds.
 4. System calculates the suggested replenishment quantity.
 5. System displays the recommendation.
-

19. Model Requirements

The machine-learning component shall:

1. Accept prepared historical data.
 2. Generate relevant features.
 3. Train candidate models.
 4. Validate models using time-aware evaluation.
 5. Compare performance.
 6. Optimize the selected model.
 7. Save the final model.
 8. Generate future predictions.
 9. Track model version information.
 10. Support future retraining.
-

20. Model Evaluation

20.1 Mean Absolute Error (MAE)

Measures the average absolute difference between actual and predicted demand.

$$\text{MAE} = \Sigma |\text{Actual} - \text{Predicted}| / n$$

Lower MAE indicates smaller average prediction error.

20.2 Root Mean Squared Error (RMSE)

Measures prediction error while giving greater importance to larger errors.

$$\text{RMSE} = \sqrt{[\Sigma(\text{Actual} - \text{Predicted})^2 / n]}$$

20.3 Mean Absolute Percentage Error (MAPE)

Measures prediction error as a percentage where appropriate.

$$\text{MAPE} = (100/n) \times \Sigma |(\text{Actual} - \text{Predicted}) / \text{Actual}|$$

MAPE should be used carefully when actual demand values are zero or very small.

21. Security Requirements

The system shall:

- Authenticate users before accessing protected features.
 - Implement role-based authorization.
 - Validate uploaded files.
 - Protect sensitive datasets.
 - Secure database credentials.
 - Use HTTPS in production.
 - Secure API endpoints.
 - Avoid exposing sensitive information in error messages.
 - Maintain logs for important system activities.
-

22. System Requirements

22.1 Software Requirements

Development:

- Python 3.11+
- Node.js
- React
- FastAPI
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- PostgreSQL/MySQL
- Git/GitHub
- VS Code

Deployment:

- Docker
- AWS/cloud infrastructure
- Cloud database
- Cloud-hosted frontend
- Cloud-hosted backend

22.2 Hardware Requirements

Minimum:

- Dual-core/quad-core processor
- 8 GB RAM

- 10 GB available storage
- Stable internet connection

Recommended:

- 16 GB RAM
 - SSD storage
 - Modern multi-core processor
-

23. Assumptions

The project assumes:

1. Historical ATM data is available.
 2. Historical data is sufficiently representative for modeling.
 3. Required fields are available or can be derived.
 4. ATM identifiers remain consistent.
 5. Data is reasonably accurate.
 6. Holiday/calendar information can be incorporated if required.
 7. Prediction performance depends on data quality and quantity.
 8. Forecasts are decision-support outputs rather than guarantees.
 9. Actual replenishment decisions remain under human/business control.
-

24. Constraints

The project may face:

- Limited historical data
 - Missing or inconsistent records
 - Changing customer behavior
 - Unexpected events
 - ATM-specific demand differences
 - Seasonal variations
 - Dataset imbalance
 - Limited real-time data
 - Computational limitations
 - Model drift
-

25. Risks and Mitigation

Risk	Impact	Mitigation
Poor data quality	High	Data validation and cleaning
Insufficient historical data	High	Feature engineering and additional data
Demand spikes	Medium/High	Outlier and event analysis
Model overfitting	High	Time-aware validation
Data leakage	High	Chronological train/test splitting
Model drift	Medium	Periodic monitoring and retraining
API failure	Medium	Error handling and logging
Unauthorized access	High	Authentication and RBAC

26. Acceptance Criteria

Data

- Dataset can be uploaded successfully.
- Invalid data is detected.
- Missing and duplicate records are handled.
- Model-ready data can be generated.

EDA

- ATM demand trends can be visualized.
- ATM-wise analysis is available.
- Seasonal and weekly patterns can be analyzed.
- Statistical metrics are available.

Machine Learning

- At least one forecasting model is trained.
- Candidate models can be compared.
- Evaluation metrics are generated.
- The final model generates future predictions.

Integration

- The model is accessible through the backend API.
- Frontend communicates successfully with the API.
- Forecasts are displayed on the dashboard.

Results

- Actual vs predicted demand can be visualized.
- Prediction errors can be analyzed.
- Model-performance metrics are displayed.
- Replenishment insights can be generated based on defined business rules.

27. Success Metrics

The following KPIs may be monitored:

KPI	Purpose
Forecast Error	Measures prediction quality
MAE	Average absolute error
RMSE	Measures larger errors

MAPE	Percentage error where appropriate
Cash-out Risk	Identifies potential shortage situations
Replenishment Efficiency	Measures operational improvement
Forecast Coverage	Measures prediction availability across ATMs

The proposed $\geq 85\%$ forecast accuracy can be treated as a project target. Final performance should be reported using clearly defined evaluation metrics and a documented test period.

28. Future Enhancements

Future versions may include:

1. Real-time ATM transaction streaming.
 2. Integration with bank/ATM management systems.
 3. Automated model retraining.
 4. Deep-learning forecasting models.
 5. Weather and event-based demand prediction.
 6. Holiday and salary-cycle analysis.
 7. Geographical demand clustering.
 8. Advanced anomaly detection.
 9. Mobile application for cash-management teams.
 10. Email/SMS notifications.
 11. Explainable AI.
 12. Multi-bank deployment.
 13. ATM replenishment route optimization.
 14. Probabilistic forecasting and confidence intervals.
-

29. Project Deliverables

The project will produce:

1. Cleaned and processed dataset
 2. EDA notebook/report
 3. Feature-engineering pipeline
 4. Trained forecasting model
 5. Model evaluation report
 6. Prediction pipeline
 7. FastAPI backend
 8. React dashboard
 9. Database
 10. API documentation
 11. Docker configuration
 12. SRS documentation
 13. Final presentation
 14. Working project demonstration
-

30. Technology Stack

Layer	Technology
Programming	Python
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn
Forecasting	ML / Time-Series Models
Backend	FastAPI
Frontend	React

Database	PostgreSQL / MySQL
----------	--------------------

Authentication	OAuth2 / JWT
----------------	--------------

Containerization	Docker
------------------	--------

Cloud	AWS
-------	-----

Version Control	Git / GitHub
-----------------	--------------

31. Requirement Traceability Matrix

Requirement Group	EPIC	Module
FR-01 – FR-08	EPIC 1	Data Pipeline
FR-09 – FR-16	EPIC 2	Analytics / EDA
FR-17 – FR-25	EPIC 3	ML + API
FR-26 – FR-33	EPIC 4	Prediction + Dashboard

32. Conclusion

ATM DemandIQ is a data-driven ATM cash-demand forecasting and decision-support system.

The system follows the complete workflow:

Data Collection → Preprocessing → EDA → Feature Engineering → Model Development → Optimization → API Integration → Forecasting → Evaluation → Replenishment Insights

By combining data analytics, machine learning, backend APIs, database management, and an interactive dashboard, ATM DemandIQ aims to provide ATM operators with useful information for planning cash replenishment.

The system is intended to support rather than replace operational decision-making. Its effectiveness will depend on data quality, forecasting performance, operational thresholds, and the environment in which it is deployed.
